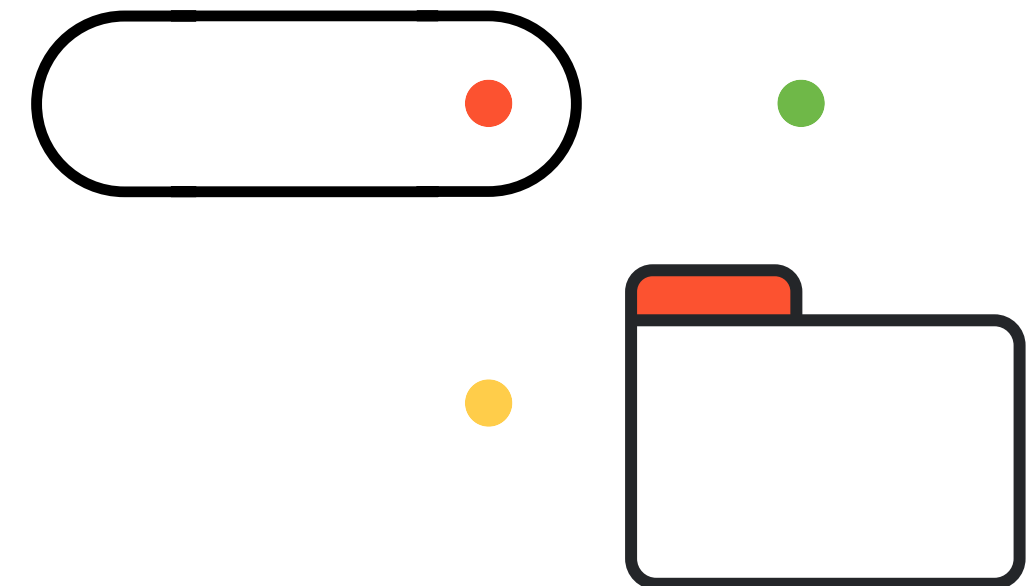


서버 비용 분석 및 최적화 도구



클라우드 컴퓨팅

인터넷을 통해 서버, 저장공간, 소프트웨어 같은 IT 자원을 직접
구축하지 않고 빌려서 사용하는 기술

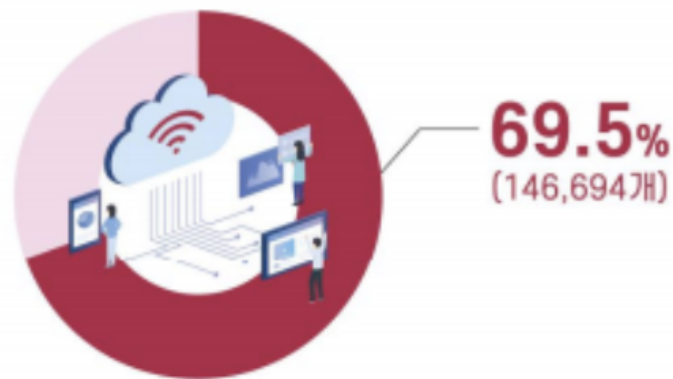




Google Cloud Platform

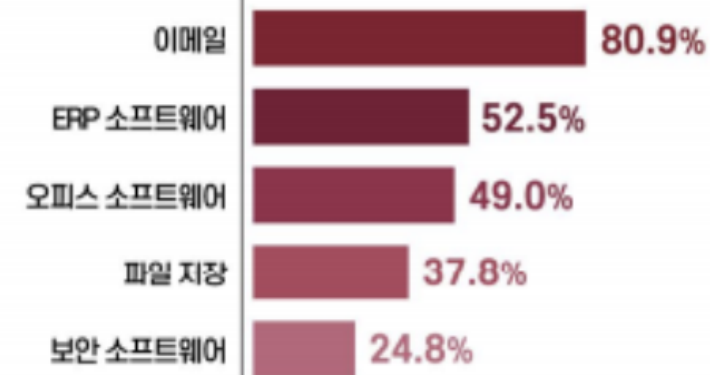
클라우드 컴퓨팅 서비스

클라우드 컴퓨팅 서비스 이용률



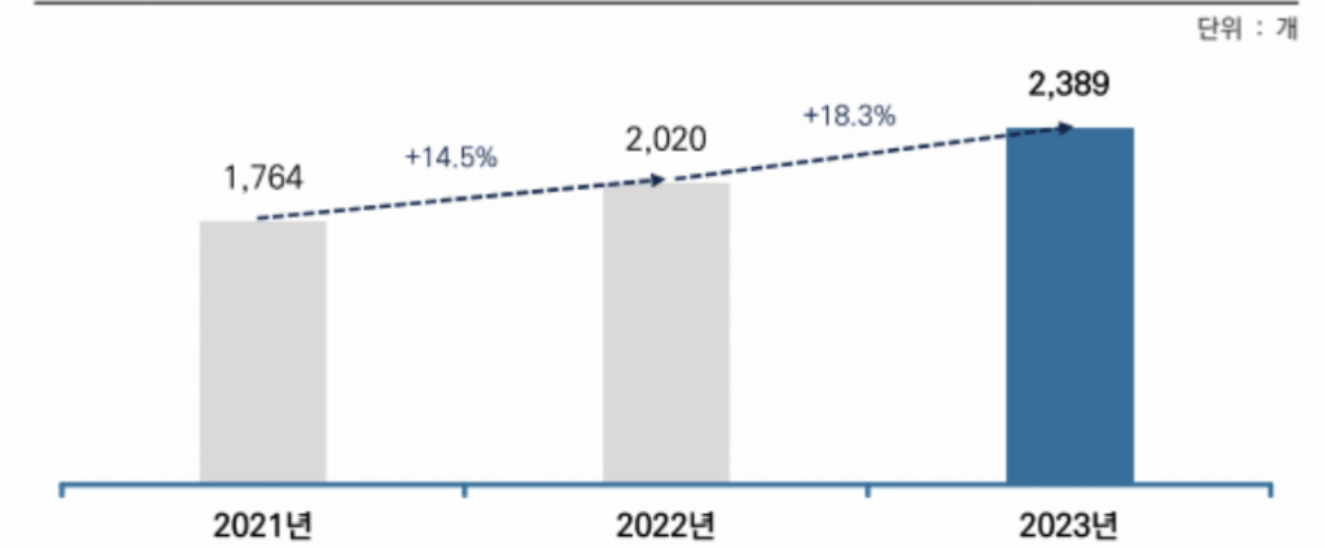
기준 : 전국의 종사자수 10인 이상 기업체

클라우드 컴퓨팅 서비스 이용분야(5순위, 복수응답)



기준 : 2022년 12월, 클라우드 컴퓨팅 서비스 이용 기업체

〈그림 2〉 클라우드 서비스 공급기업 수 3개년 추이



〈표 3〉 3개년 클라우드 서비스 공급기업 수

단위 : 개

구분	2021년	2022년	2023년
기업 수	1,764	2,020	2,389
증가 수	+355	+256	+369
증가율(%)	+25.2%	+14.5%	+18.3%

Over half of companies say their cloud costs are too high in 2024

CloudZero's survey found that less than half of companies report healthy cloud costs in 2024, with 58% of respondents saying their cloud costs are too high.

Most concerning, the data shows an increase in the number of companies saying their costs are "way too high" — from 11% in 2022 to 14% in 2024. This isn't a huge jump, but it indicates a continued lack of control over cloud costs.

Nine out of 10 employees say lack of cloud cost visibility prevents them from doing their job well

89% of respondents said that lack of cloud cost visibility has an impact on their ability to carry out their role, with just under half reporting a significant impact.

Broken down by role, 22% of engineers said cloud costs cause high levels of disruption to their work (the equivalent of a full sprint), doubling since 2022 (11%). Meanwhile, 23% of finance professionals reported high levels of disruption, pointing to this being an equal problem for both teams.



news

비용 높음

2024년 CloudZero의 조사에 따르면, 58%의 기업이 자사의 클라우드 비용이 '너무 높다'고 응답했습니다.

비용 가시성 부족

2024년 CloudZero 조사에서 89%의 직원이 클라우드 비용 가시성 부족이 업무에 영향을 준다고 답했습니다.
엔지니어와 재무팀 모두 비용 관리의 어려움을 호소하고 있습니다.

① 사용하지 않은 리소스

클라우드는 개발 속도에 중점을 두고 있기 때문에 새로운 인스턴스를 생성하거나 가동 중인 인스턴스를 계속 실행하도록 내버려 두기가 매우 쉽습니다. 즉, 가상머신과 스토리지를 유휴 상태로 두는 경우가 많습니다. 주로 스테이징, 테스트, QA 등에 사용하는 개발 환경이 이에 해당합니다. 연결되지 않은 스토리지, 사용하지 않는 스냅샷, 사용되지 않지만 실행 중인 리소스 등으로 인해 비용을 지불하고 있습니다. 유휴 리소스가 운영되고 있다면 유휴 비용이 발생한다는 것입니다.

② 컴퓨팅 리소스

RightScale 조사에 따르면, 평균 클라우드 고객 청구서의 35% 가량이 낭비되는 것으로 보고 있습니다^[3]. 클라우드 환경에서 애플리케이션을 운영하기 위해 CPU, 메모리 등을 할당하고 관리하는 것은 매우 일반적입니다. 하지만 클라우드 리소스가 과도하게 할당되거나 예약한 리소스를 제대로 사용하지 않는 경우에도 비용이 발생할 수 있습니다. 실제로 필요한 리소스 이상으로 리소스를 높게 잡는 경우가 종종 발생합니다. 이는 사용하지 않는 리소스에 대해서도 비용을 지불해야 합니다. 인스턴스의 40% 정도가 워크로드에서 필요로 하는 리소스보다 한 사이즈 크게 잡혀 있다고 합니다. 만일을 대비한 과잉 프로비저닝 습관은 곧바로 고치기 어려울 수 있습니다. 이외에도 비효율적인 컨테이너, 활용도가 낮은 데이터베이스 등으로 인해 클라우드 낭비가 발생하기도 합니다.

하지만, 비용 절감이라는 초기의 목표와 달리 현실에서는 지출을 낭비하는 현상이 발생하고 있습니다. 안타깝지만 기업이 클라우드로 전환할 때 클라우드 비용에 대한 절감 효과가 바로 나타나지 않을 수도 있습니다. 실제로 일부 기업에서는 클라우드 모델을 적용했지만 온프레미스 시스템보다 월별 지출이 더 많아진 경우가 있었습니다. 클라우드가 확장함에 따라 클라우드 낭비도 확대되고 있었습니다. Accenture에 의하면, 평균적으로 기업들은 클라우드 지출의 30%를 낭비하는 것으로 나타났습니다^[2]. 최근 1년 간 클라우드 사용량이 급격히 증가하고 현실적으로 지출이 증가하면서 클라우드 비용 최적화는 핫한 주제가 되었습니다.



news

유휴 리소스 사용 / 필요 이상의 사이즈 이용

Accenture에 의하면, 평균적으로 기업들은 클라우드 지출의 30%를 낭비

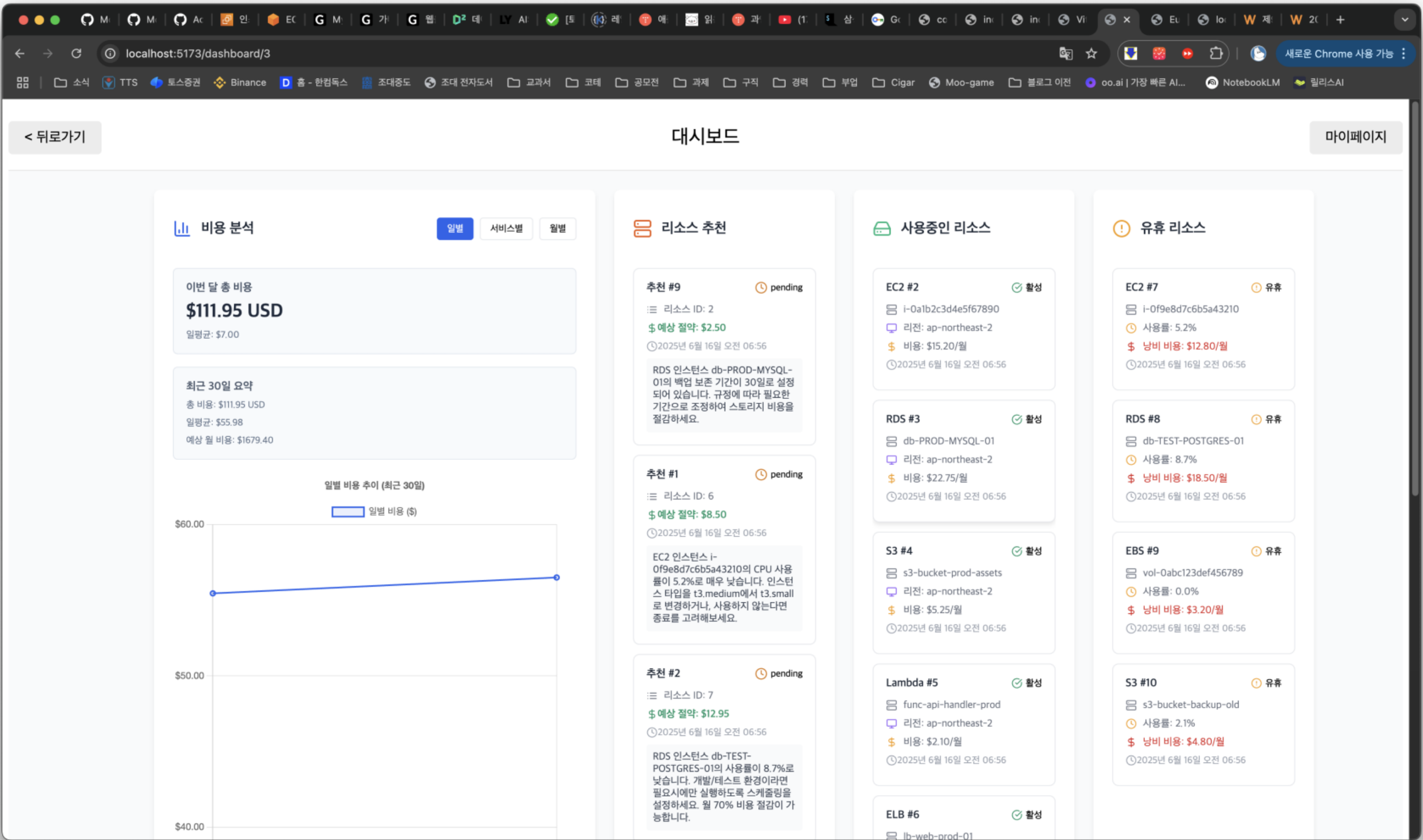


유휴 리소스 탐지

인스턴스 사이즈 추천

가시성 개선

통합 클라우드 관리



비용 분석

이번 달 총 비용
\$11.95 USD
 일평균: \$7.500

서버스탑 비용 분포

리소스 유형	비용 (USD)
EC2	~8.00
EBS	~3.50
S3	~0.45

리소스 추천

추천 사항이 없습니다

사용중인 리소스

사용중인 리소스가 없습니다

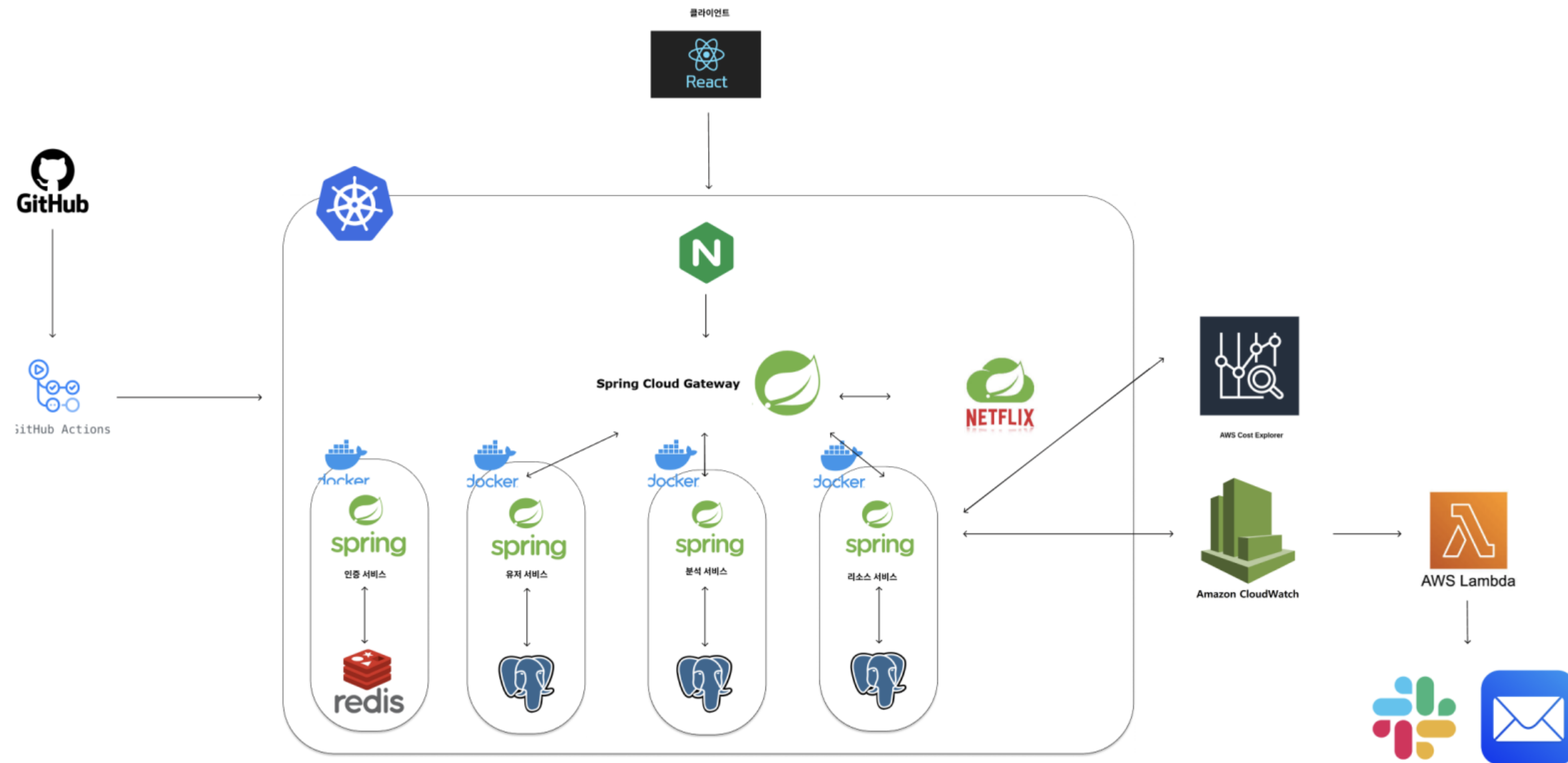
유류 리소스

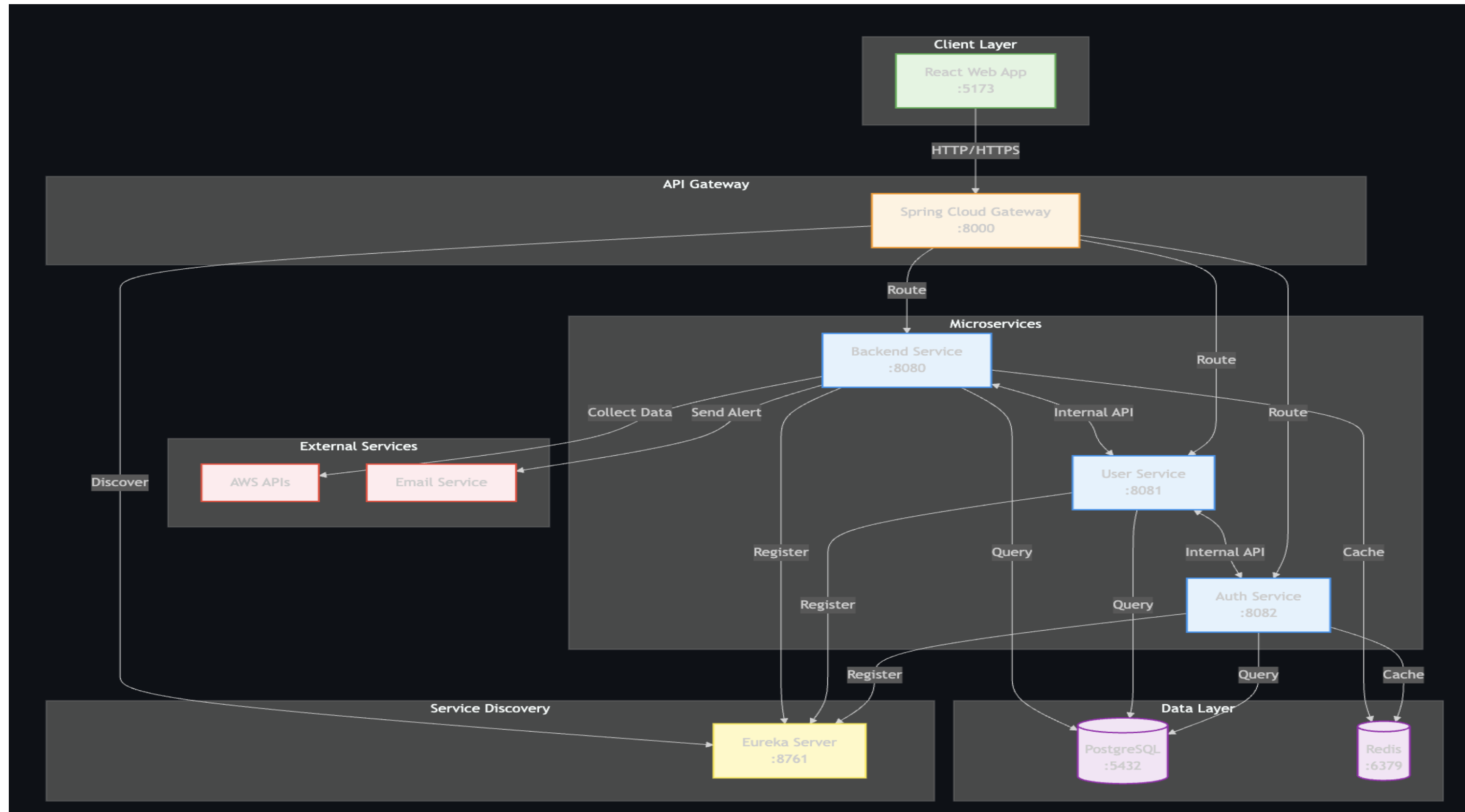
유류 리소스가 없습니다

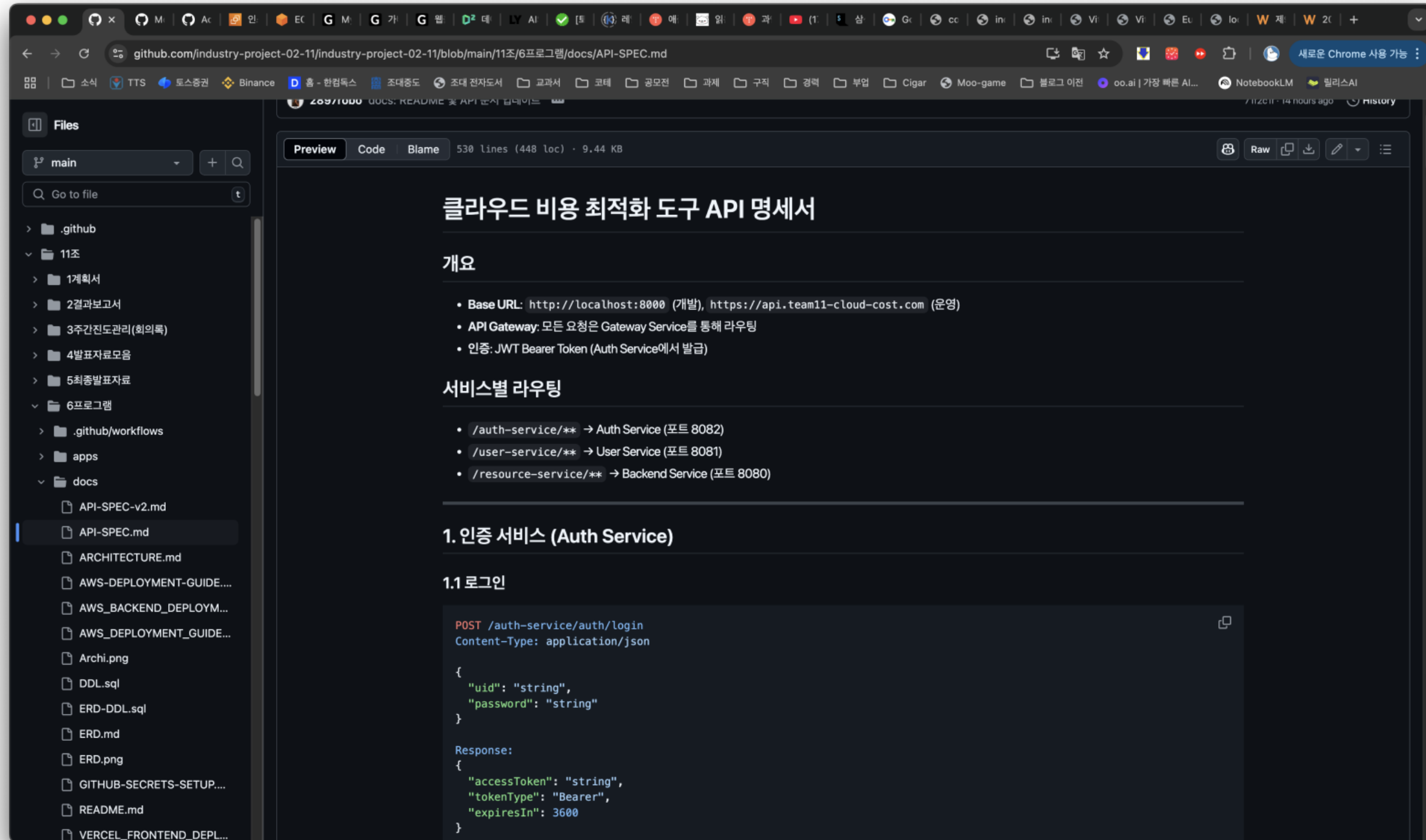


차별점

항목	기존 솔루션 (AWS Cost Explorer)	제안 도구
분석 포인트	비용 위주	리소스 사용량 기반 절감 포인트까지 분석
실시간 대응	x	리소스 급등 시 알림 전송
사용자 친화성	복잡한 UI, 설정 필요	간편한 설정
다양한 클라우드 지원	x	o









Files

main

Go to file

.github

11조

1계획서

2결과보고서

3주간진도관리(회의록)

4발표자료모음

5최종발표자료

6프로그램

.github/workflows

apps

docs

API-SPEC-v2.md

API-SPEC.md

ARCHITECTURE.md

AWS-DEPLOYMENT-GUIDE.....

AWS_BACKEND_DEPLOYM...

AWS_DEPLOYMENT_GUIDE...

Archi.png

DDL.sql

ERD-DDL.sql

ERD.md

ERD.png

GITHUB-SECRETS-SETUP....

README.md

VERCEL_FRONTEND_DEPL...

industry-project-02-11 / 11조 / 6프로그램 / docs / ARCHITECTURE.md

PreviewCodeBlame272 lines (224 loc) · 6.39 KB

RawDownloadEdit

JWT 토큰리스트

임시 데이터 저장

성능 최적화

데이터 흐름

1. 인증 플로우

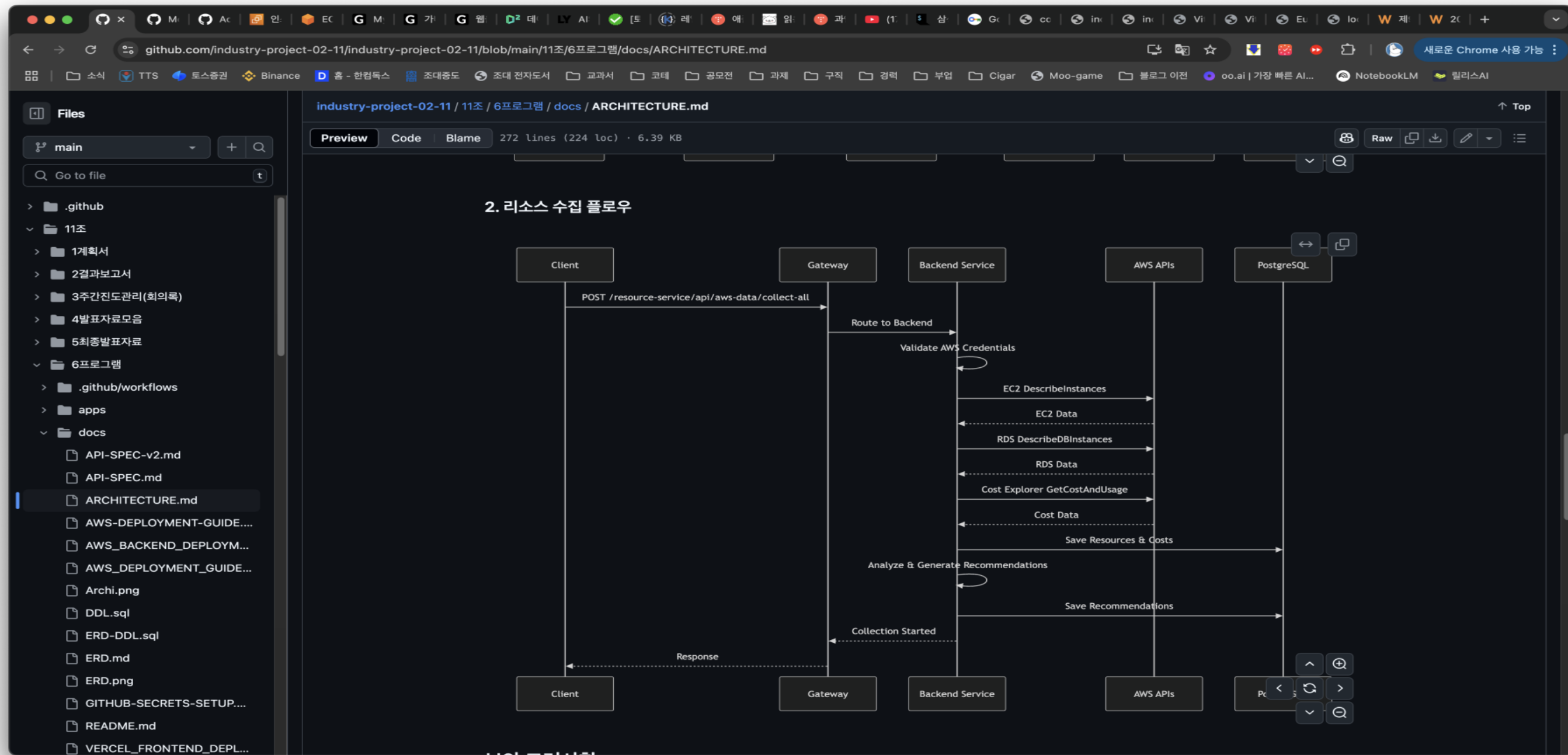
```
sequenceDiagram
    participant Client
    participant Gateway
    participant Auth Service
    participant User Service
    participant Redis
    participant PostgreSQL

    Client->>Gateway: POST /auth-service/auth/login
    activate Gateway
    Gateway->>Auth Service: Route to Auth Service
    deactivate Gateway
    activate Auth Service
    Auth Service->>User Service: Validate User (Internal API)
    deactivate Auth Service
    activate User Service
    User Service->>PostgreSQL: Query User
    deactivate User Service
    activate PostgreSQL
    PostgreSQL-->>User Service: User Data
    deactivate PostgreSQL
    deactivate User Service
    User Service-->>Auth Service: User Valid
    deactivate User Service
    activate Auth Service
    Auth Service->>Redis: Store Session
    deactivate Auth Service
    activate Redis
    Redis-->>Auth Service: 
    deactivate Redis
    deactivate Auth Service
    Auth Service-->>Gateway: JWT Token + Refresh Token
    deactivate Auth Service
    activate Gateway
    Gateway-->>Client: Response with Tokens
    deactivate Gateway
```

2. 리소스 수집 플로우

```
sequenceDiagram
    participant Client
    participant Gateway
    participant Backend Service
    participant AWS APIs
    participant PostgreSQL

    Client->>Gateway: POST /resource-service/api/aws-data/collect-all
    activate Gateway
    Gateway->>Backend Service: 
    deactivate Gateway
    activate Backend Service
    Backend Service->>AWS APIs: 
    deactivate Backend Service
    activate AWS APIs
    AWS APIs-->>Backend Service: 
    deactivate AWS APIs
    deactivate Backend Service
    Backend Service->>PostgreSQL: 
    deactivate Backend Service
    activate PostgreSQL
    PostgreSQL-->>Backend Service: 
    deactivate PostgreSQL
    deactivate Backend Service
    Backend Service-->>Gateway: 
    deactivate Backend Service
    activate Gateway
    Gateway-->>Client: 
    deactivate Gateway
```



Files

main

Go to file

> .github

> 11조

> 1계획서

> 2결과보고서

> 3주간진도관리(회의록)

> 4발표자료모음

> 5최종발표자료

> 6프로그램

> .github/workflows

> apps

> docs

API-SPEC-v2.md

API-SPEC.md

ARCHITECTURE.md

AWS-DEPLOYMENT-GUIDE...

AWS_BACKEND_DEPLOYM...

AWS_DEPLOYMENT_GUIDE...

Archi.png

DDL.sql

ERD-DDL.sql

ERD.md

ERD.png

GITHUB-SECRETS-SETUP....

README.md

VERCEL_FRONTEND_DEPL...

Preview

Code

Blame

197 lines (173 loc) · 5.36 KB

Raw

Download

Copy

Share

클라우드 비용 최적화 도구 ERD

Entity Relationship Diagram

The diagram illustrates the database structure for a cloud cost optimization tool. It features several entities and their relationships:

- USERS**: Contains fields like serial, id, email, password_hash, name, created_at, and updated_at. It is the central entity, connected to others via 'owns', 'has', 'manages', and 'performs' relationships.
- AWS_ACCOUNTS**: Contains fields like serial, id, user_id, account_alias, aws_account_id, access_key_id, secret_access_key, region, is_active, created_at, and updated_at. It is 'owned' by a user and 'contains' resources.
- CONFIGS**: Contains fields like serial, id, user_id, idm_threshold, budget_limit, alert_enabled, created_at, and updated_at. A user 'has' configurations.
- RESOURCES**: Contains fields like serial, id, user_id, aws_resource_id, resource_name, service_type, instance_type, region, is_idm, usage_rate, memory_usage_rate, network_usage_rate, cost_daily, cost_monthly, status, tags, created_at, and updated_at. It is 'generated' by AWS accounts and 'has' cost histories.
- COST_HISTORIES**: Contains fields like serial, id, user_id, aws_account_id, service_name, resource_type, cost, cost_id, currency, usage_date, usage_quantity, usage_unit, row_data, created_at, and updated_at. It is 'has' by resources.
- ALERTS**: Contains fields like serial, id, user_id, alert_type, title, message, severity, is_read, channel, and metadata. A user 'manages' alerts.
- ALERT_LOGS**: Contains fields like serial, id, user_id, action, target_type, target_id, is_address, user_agent, request_method, request_url, response_status, meta, and created_at. A user 'performs' alert logs.



kanban



github.com/orgs/industry-project-02-11/projects/1

industry-project-02-11 / Projects / kanban board

kanban board

Monthly roadmap Backlog + New view

Filter by keyword or by field

April 2025 May 2025

Markers Sort Date fields Month Today < >

No Team 23

Mon, Mar 10 - Wed, Jun 18

- 주제 선정 및 팀 구성 완료 #2
- 산학프로젝트 참여 신청서 및 계획서 제출 #3
- GitHub 레포지토리 생성 및 초기 폴더 구조 세팅 #4
- 역할 분담 및 개발 일정 수립 #5
- 기술 스택 및 아키텍처 논의 #6
- ERD 설계 및 공유 (dbdiagram 기반) #7
- ERD 기반 DDL 작성 (ERD-DDL.sql) #8
- Spring Boot 프로젝트 생성 및 디렉토리 정리 #9
- GitHub Actions CI 환경 설정 (Gradle + Te... #10
- Docker 기반 PostgreSQL / Redis 로컬 테스... #11
- application.yml 환경 분리 (dev / prod) #12
- JPA 기반 Entity 클래스 구현 시작 (User, Re... #13
- Flyway 기반 초기 마이그레이션 적용 준비 #14
- Spring Security + JWT 인증 구조 설계 #15
- 회원가입 및 로그인 API 구현 #16

application.yml 환경 분리 (dev / prod) #12

JPA 기반 Entity 클래스 구현 시작 (User, Resource 등) #13

Flyway 기반 초기 마이그레이션 적용 준비 #14

Spring Security + JWT 인증 구조 설계 #15

회원가입 및 로그인 API 구현 #16